11c — SOA (Service-Oriented Architecture)

Key Concepts

- **SOA** — architectural style where services communicate through a central Enterprise Service Bus (ESB)
- **ESB (Enterprise Service Bus)** — central communication layer that handles routing, transformation, protocol translation, and security
- **Smart pipes, dumb endpoints** — SOA philosophy; ESB does the heavy lifting
- **Smart endpoints, dumb pipes** — microservices philosophy; the opposite of SOA
- Dominant enterprise pattern in the **2000s–2010s**

How It Works

```
Service A ──┐
Service B ──┤
Service C ──┼──► Enterprise Service Bus (ESB) ──► Routes to correct service
Service D ──┤
Service E ──┘
```

- All communication goes **through the ESB** — services never talk directly
- ESB handles: routing, protocol translation, data transformation, security, logging

ESB Responsibilities

Responsibility	What It Does
Routing	Sends messages to the correct service
Protocol translation	REST ↔ SOAP translation between services
Data transformation	XML ↔ JSON conversion
Security	Central auth enforcement
Logging & monitoring	All traffic passes through — central visibility

SOA vs Microservices

	SOA	Microservices
Communication	Through central ESB	Direct (REST, gRPC, messaging)
Pipe philosophy	Smart pipes, dumb endpoints	Smart endpoints, dumb pipes
Service size	Larger, coarser-grained	Small, fine-grained
Data	Services often share a database	Each service owns its own DB
Technology	SOAP, XML, WSDL	REST, gRPC, JSON

	SOA	Microservices
Typical use	Large enterprises, legacy systems	Modern cloud-native apps

Azure Service Bus vs ESB

	Azure Service Bus	ESB (SOA)
Purpose	Async message delivery — pub/sub	Central nervous system
Intelligence	Dumb pipe — just delivers messages	Smart pipe — transforms, routes, enforces rules
Business logic	None — stays in services	Leaks into ESB over time
Failure impact	Only async messaging affected	Everything stops

The ESB Problem

- **Single point of failure** — ESB goes down → entire system goes down
- **Performance bottleneck** — all traffic must pass through one layer
- **Business logic creep** — teams gradually push logic into ESB → unmaintainable
- Microservices evolved from SOA specifically to fix these problems

Industry Examples

- **Banks, insurance, government** — still run SOA with massive legacy ESB investments
- **Microsoft BizTalk** — Microsoft's ESB product, common in enterprise .NET shops
- **IBM MQ, MuleSoft** — other popular ESB products

Interview Questions

Q: What is SOA? A: An architecture where services communicate through a central Enterprise Service Bus that handles routing, transformation, and protocol translation.

Q: What's the difference between SOA and microservices? A: SOA has smart pipes (ESB does heavy lifting), dumb endpoints. Microservices have smart endpoints, dumb pipes — services communicate directly. Microservices also enforce database-per-service ownership.

Q: What's the problem with ESB? A: It becomes a single point of failure, a performance bottleneck, and teams gradually push business logic into it making it impossible to maintain.